Structured Planning Under Uncertainty: A Decision Graph Approach to AI-Assisted Software Development

Omkar Khade Naman Sogani Ashvin Sehgal Mari Garey
Elena Motonishi Sreemae Konduru

University of Wisconsin–Madison
CS 639: Deep Learning for NLP, Spring 2026
Github repository: https://github.com/sogani12/llm-planning-graph

Abstract

Large language models are increasingly used as software planning partners, but their outputs are often optimized for the immediate prompt rather than for consistency with project-wide design intent. In practice, this means that assumptions, architectural decisions, constraints, and risks that appear during a planning conversation are quickly lost. We address this problem with a typed decision graph that extracts structured planning state from developer–AI interactions and related project documents, then uses that state to support incremental plan maintenance and proactive risk surfacing. Our system combines a schema of software-planning entities, an LLM-based extraction pipeline, heuristic and graph-based update mechanisms, a small corpus analysis of design-heavy software artifacts, and a prefix-tuning module for repo-aware specialization. Across four current planning projects in our evaluation suite, graph-augmented planning consistently improves structural completeness and usually improves functional quality as well. The main contribution of this report is an end-to-end account of the method, implementation, and current empirical findings, together with a candid discussion of limitations and next steps.

1 Introduction

AI coding assistants are now routinely asked to plan software systems, propose architectures, select libraries, and sequence implementation steps. However, most current interactions are stateless at the level of project reasoning. A model may produce a plausible answer to the current request, yet fail to preserve earlier design choices, hidden assumptions, or cross-module dependencies. In software engineering settings, this limitation matters because planning errors are often not local: one weak assumption or poorly justified decision can invalidate downstream components and only surface much later in development.

This project studies a simple question: can we make planning state explicit enough that an AI assistant can reason over it, maintain it, and expose its failure modes? Our answer is to represent project planning as a typed directed graph. Instead of treating a planning conversation as disposable text, we extract nodes for objectives, requirements, assumptions, decisions, components, interfaces, risks, and tests, along with typed edges that connect them. The resulting graph serves as a persistent representation of architectural intent. In our design, the graph is not merely a logging artifact. It becomes a working memory that supports incremental updates, invalidation propagation, and proactive surfacing of risks tied to uncertain assumptions or poorly tested components.

The motivation comes from a practical gap we observed in AI-assisted software development. Existing coding assistants are strong at producing localized plans, code snippets, and tool calls, but weak at preserving global consistency over longer planning trajectories. They often bury critical assumptions inside prose, omit rejected alternatives, and fail to identify how one decision constrains future steps. This creates an accountability problem as well as a reasoning problem: when a plan goes wrong, it is difficult to inspect which earlier choices caused the failure.

Our work is related to recent efforts in graph-

augmented reasoning and retrieval, but differs in emphasis. Prior work has shown that graph structure can improve multi-hop reasoning and knowledge-intensive generation, and that LLM systems benefit from explicit planning or iterative retrieval. Our setting is narrower and more applied: software project planning, where the target structure is not a static knowledge graph but a living representation of design intent built from conversation and documentation. We care not only about whether a model can answer a question, but whether it can preserve and revise the rationale behind a plan as the project evolves.

This report makes four contributions. First, we define a lightweight but expressive decision graph schema for software planning. Second, we implement an extraction and maintenance pipeline that can create graphs from text and update them incrementally as new information arrives. Third, we ground the design with an exploratory corpus study over GitHub issues, Stack Overflow discussions, and software post-mortems to understand what linguistic signals are available for assumptions, decisions, dependencies, and risks. Fourth, we present an initial evaluation across four planning tasks comparing a standard planning condition against a graph-augmented condition. The current results suggest that the graph consistently improves explicitness, audibility, and risk awareness, even when the planner still makes some incorrect decisions.

2 Related Work

Our project sits at the intersection of LLM planning, graph-augmented reasoning, and software-engineering assistance.

Graph Chain-of-Thought augments LLM reasoning with graph-structured evidence instead of isolated retrieved passages (Jin et al., 2024). Its core insight is that structured relations can improve multi-step reasoning and reduce hallucination on knowledge-intensive tasks. We build on a similar intuition, but our graph is extracted from planning discourse itself rather than from an external knowledge graph.

Search-R1 trains LLMs to decide when and how to issue search queries during multi-step reasoning (Jin et al., 2025). That work shows that adaptive retrieval can substantially strengthen reasoning quality. In contrast, our method does not focus on external search. Instead, it structures the model's internal planning context so that future decisions can reference earlier assumptions, risks, and dependencies in a persistent form.

GNN-RAG combines graph neural retrieval with LLM reasoning on knowledge graphs (Mavromatis and Karypis, 2025). It demonstrates that retrieving relevant subgraphs can outperform or match much larger black-box systems on multi-hop reasoning. Our work differs in two important ways: our graphs are generated from software-planning artifacts rather than provided as a pre-built knowledge graph, and we care about maintenance over time, not only retrieval at inference.

Retrieve-Plan-Generation proposes an iterative framework in which retrieval and explicit planning improve knowledge-intensive generation (Lyu et al., 2024). This paper is particularly relevant because it treats planning as an intermediate structure rather than a hidden latent behavior. Our approach is similar in spirit, but our intermediate representation is a typed decision graph designed for software architecture reasoning and change propagation.

Finally, recent surveys of LLM planning emphasize that long-horizon consistency, executable structure, and evaluation methodology remain open problems (Wei et al., 2025). Our project can be viewed as one concrete response to those challenges: we externalize planning state into a graph and evaluate both the structural quality of that graph and its downstream effect on planning outputs.

Overall, prior work supports the value of explicit structure, iterative reasoning, and graph-based context. The main gap is that little of this work directly addresses the software-planning setting where assumptions, decisions, and risks must be revised over time. Our method aims to fill that gap with a persistent, typed representation tailored to developer–AI collaboration.

3 Proposed Method

3.1 Decision graph schema

We represent planning state as a typed directed graph implemented with Pydantic models and a NetworkX backend. The current schema defines eight node types: *objective*, *requirement*, *assumption*, *decision*, *component*, *interface*, *risk*, and *test*. These nodes capture both high-level intent and lower-level implementation structure. For example, *assumption* nodes store a confidence score and validation flag, *decision* nodes store rationale and alter-

natives considered, and *risk* nodes store a severity value. This design allows the graph to encode not only what the plan is, but how certain the team is about it and what tradeoffs were considered.

Edges express typed relations among nodes. The implementation currently supports eleven edge types: *motivated_by*, *assumes*, *implements*, *depends_on*, *conflicts_with*, *invalidates*, *exposes*, *consumes*, *verifies*, *guards_against*, and *validates*. This already exposes the richer edge inventory, and it matters because software planning often requires us to connect a design decision not just to an objective, but also to the assumptions it depends on, the risks it creates, and the tests that can falsify it.

3.2 Graph extraction from text

The first stage of the system converts unstructured text into a graph. In the current implementation, the extractor prompts an LLM to read a completed developer–AI conversation or design document and emit valid JSON matching the graph schema. The prompt explicitly requests node and edge types, significant decisions with rationale and alternatives, implicit assumptions, and notable risks. The extractor uses `claude-sonnet-4-6` by default and allocates up to 4096 output tokens. If the model returns malformed JSON, the system retries once with a repair prompt. This makes the extraction pipeline more robust without requiring a separate parser.

The extraction stage is intentionally schema-driven. By forcing the model to emit a typed graph rather than free-form prose, we make downstream evaluation and maintenance possible. The cost of this design is that extraction quality depends on both prompt quality and the model's ability to remain faithful to the schema.

3.3 Incremental maintenance and invalidation

A static graph is useful, but our main goal is a living planning representation. The maintenance module processes new conversation turns or update text against an existing graph. It first tries to interpret the update as graph JSON. If that fails, it either calls the extractor or falls back to a lightweight heuristic graph builder based on keywords such as *must*, *assume*, *decide*, or *risk*. Nodes are merged by normalized type-label pairs. When merging, the system preserves longer descriptions, unions list-valued attributes, and upgrades boolean or scalar fields when new evidence is stronger.

The maintenance stage also supports invalidation. If an update introduces an *invalidates* edge or contains invalidation cues such as "no longer" or "replaced," the system computes the blast radius by traversing descendants along invalidation edges. This mechanism formalizes a key intuition behind the project: planning errors should not be treated as isolated mistakes. They should trigger structured review of the downstream decisions and requirements that depended on them.

3.4 Risk surfacing

The failure module proactively surfaces risk nodes from the current graph state. In the current implementation, it inspects unvalidated or low-confidence assumptions and components with weak test coverage. For each such item, it constructs new risk candidates and assigns heuristic severity scores based on confidence levels or dependency exposure. While simple, this mechanism demonstrates how the graph can support anticipatory reasoning. Rather than waiting for a failure to appear in a later planning turn, the system can flag likely weak points as soon as their structural signatures appear in the graph.

3.5 Corpus analysis and prefix tuning

Because extraction quality depends on whether software-planning artifacts actually contain recoverable linguistic cues, we ran an exploratory data analysis pipeline over a collected corpus. The corpus summarized in our slides contains 166 documents and 5,702 sentences from GitHub design-heavy issues and repository documents, Stack Overflow software-design discussions, and public software post-mortems. Our EDA pipeline measures hedge patterns, node-type keyword density, dependency verb patterns, and uncategorized samples. We found 320 hedge hits across the corpus, suggesting that assumptions are often signaled indirectly rather than explicitly. GitHub issues provided the strongest decision and risk signal, while some edge types remained sparse.

In parallel, we developed a repo-aware prefix-tuning module based on PEFT. The current implementation routes requests to adapters using structured metadata and trains prefix adapters with 20 virtual tokens, prefix projection enabled, learning rate 10^{-4}, batch size 2, gradient accumulation 8, and sequence length up to 2048 tokens. This module is intended to specialize behavior across repository domains while leaving knowledge retrieval to

the graph and surrounding context. At the time of writing, the prefix-tuning system is implemented and partially bootstrapped, but it is less mature than the extraction and evaluation pipeline.

4 Experimental Settings

4.1 Datasets and tasks

We use two kinds of data. The first is the EDA corpus used to motivate the schema and extraction design. It includes design-heavy GitHub issues from eight repositories, 30 Stack Overflow questions tagged `software-design` with their top answers, and a curated post-mortem collection. The second is a planning-task evaluation setup in which we compare a standard planning condition against a graph-augmented condition on realistic software-design problems.

For the results reported in this paper, we focus on the Job Copilot Pipeline case study presented in our slide deck. This task asks the planner to design an automated job-application system that scrapes boards, handles authentication, generates tailored resumes, and recovers gracefully from failures. We use this case study because it is the most complete end-to-end example in the current project artifacts and includes both rubric-based comparison and a graph visualization extracted from the planning interaction.

4.2 Comparison conditions

Our main comparison is between two planning conditions. **Condition A** is a standard LLM planning session in which the model receives the prompt and produces a plan with no explicit graph feedback. **Condition B** is graph-augmented: planning exchanges are converted into a decision graph, and the graph is fed back as structured planning context. The intended role of Condition B is not merely to improve final answers, but to force explicit representation of decisions, assumptions, alternatives, and risks.

The slide deck also proposes a RAG-only baseline to isolate the value of graph structure from retrieval alone. That baseline is part of the planned evaluation design, but it is not yet implemented in the current repository. We therefore frame the present results as a two-condition study with a future extension.

4.3 Metrics

We evaluate at two levels. First, we use rubric-based planning scores for the case study itself. The presentation reports separate scores for *functional* quality and *structural* quality. Functional quality captures whether the produced system design addresses the task requirements, while structural quality captures whether the planning process externalizes assumptions, rationale, and risks in a form that can be inspected later.

Second, we track graph-quality metrics drawn from both the slide deck and the evaluation code: *node recall*, *edge precision*, and *failure-point recall*. Node recall measures how many ground-truth planning entities are recovered by the extracted graph after normalization by type and label. Edge precision measures how many predicted typed relations are actually correct. Failure-point recall measures whether documented failure modes are captured by extracted *risk* or *assumption* nodes, which makes it a lightweight proxy for failure detection. Alongside these metrics, we also analyze graph-specific artifacts qualitatively: whether assumptions are explicitly enumerated, whether risks are severity-rated, whether rejected alternatives are recorded, and how large the extracted graph becomes. These measures are especially important for our project because our goal is not only better answers, but more auditable planning.

4.4 Implementation details

The core graph uses a directed NetworkX representation with Pydantic v2 schema validation. The API layer is implemented with FastAPI and exposes endpoints for extraction, incremental updates, and risk surfacing. The current tests cover graph round-tripping, blast-radius traversal, update handling, risk surfacing, prompt loading, and metric behavior.

For the prefix-tuning subsystem, the default adapter configuration uses a single `graph_extractor` route with structured JSON output constraints. The bootstrapping script currently synthesizes training examples from existing graph artifacts in the evaluation suite and creates train/validation/test splits for rapid experimentation.

Metric	Condition A	Condition B
Functional score	5.5 / 7	6 / 7
Structural score	1 / 3	3 / 3
Runtime choice	Python, implicit	Python, documented + alternatives
Assumptions	None enumerated	8 confidence-rated nodes
Risks	6 informal items	9 severity-rated nodes
Graph size	—	57 nodes, 64 edges

Table 1: Job Copilot Pipeline results.

Version	Node recall	Edge precision	Failure-point recall
Job Copilot final graph	1.00	1.00	1.00

Table 2: Intrinsic graph-quality metrics for the Job Copilot Pipeline from the evaluation code.

5 Results and Analysis

5.1 Job copilot experiment

Table 1 summarizes the results for the Job Copilot Pipeline. Condition B improves both kinds of evaluation, but the gains are much larger on structural quality than on pure task completion. Functional score rises from 5.5/7 to 6/7, while structural score rises from 1/3 to 3/3. This asymmetry is one of the most important findings of the project: graph augmentation helps most when the evaluation rewards explicit reasoning structure rather than only the final recommendation.

The case study is especially informative because it shows both the strengths and the limits of the graph. In this example, both conditions selected Python for Playwright, which is the wrong runtime choice. Condition B therefore did not fix the underlying mistake. However, it made the mistake legible: the graph documented the choice, recorded alternatives, and attached explicit assumptions and risks to the decision. In other words, the graph did not guarantee correctness, but it improved auditability and challengeability.

Condition B also improves structural coverage dramatically. Condition A produced no enumerable assumptions, whereas Condition B surfaced eight explicit assumption nodes with confidence scores. Similarly, Condition A included six risks only as informal prose, while Condition B represented nine risks as typed, severity-rated nodes. This matters because many planning failures arise from hidden uncertainty rather than from immediately visible task omissions. The graph turns that hidden uncertainty into a manipulable object.

The Condition B graph contained 57 nodes and 64 edges extracted from two planning exchanges.

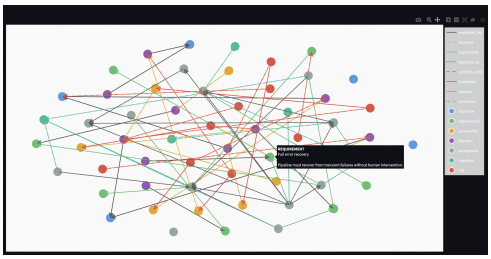

Figure 1: Decision graph for the Job Copilot Pipeline under Condition B. The graph contains 57 nodes and 64 edges extracted from two planning exchanges.

Figure 1 shows this graph. The visualization makes clear that the system is not just storing a flat checklist. It captures objectives, requirements, decisions, components, interfaces, assumptions, and risks as a connected planning structure. This connectedness is what enables later operations such as invalidation propagation and targeted risk review.

The corpus analysis helps explain these outcomes. Our EDA suggests that assumption cues are sparse and often indirect, while decision and risk signals are concentrated in design-heavy GitHub discussions. This makes assumption extraction intrinsically difficult in free-form planning prose. By explicitly requiring assumption nodes in the extraction prompt and by maintaining them over time, Condition B counteracts that sparsity. The result is a planner that is better at surfacing uncertainty, even if its first-order architectural choice is still occasionally wrong.

5.2 Content summarizer experiment

The Content Summarizer Pipeline shows a similar pattern, but in a task where compliance, source variability, and personalization play a more central role. In this experiment, Condition B again outperforms Condition A on both functional and structural criteria. According to the project rubric, Condition A achieves 6/8 functional objectives and 1/3 structural objectives, while Condition B reaches 8/8 functional objectives and 3/3 structural objectives. The final graph for Condition B contains 39 nodes and 54 edges, including 10 decision nodes, 6 assumption nodes, and 4 risk nodes.

The qualitative differences are especially revealing. Condition A already produces a relatively strong functional plan: it covers ingestion, storage, personalization, retries, and fixture-based testing. However, several important ideas remain underspecified. In the rubric notes, compliance appears mainly as a guideline rather than as an enforce-

Metric	Condition A	Condition B
Functional score	6 / 8	8 / 8
Structural score	1 / 3	3 / 3
Decision nodes	0	10
Assumption nodes	0	6
Risk nodes	0	4
Graph size	—	39 nodes, 54 edges

Table 3: Content Summarizer Pipeline results from the evaluation rubric.

Version	Node recall	Edge precision	Failure-point recall
v1	0.38	0.55	0.75
v2	0.51	1.00	1.00
v3	0.67	1.00	1.00
v4	0.82	1.00	1.00

Table 4: Intrinsic graph-quality metrics for the Content Summarizer Pipeline across iterative graph versions.

able gate, adaptive relevance has no explicit fallback when feedback is sparse, and summarization constraints are less tightly coupled to downstream digest generation. Condition B addresses these gaps by making them explicit planning objects. It introduces a daily policy-check stage with a no-ingest safe mode on failure, a thresholded feedback fallback that keeps baseline ranking until enough votes are collected, and stronger summarization guardrails tied to the digest pipeline.

This experiment also highlights a different benefit of the graph from the one emphasized in Job Copilot. In the Job Copilot case, the main gain was auditability even when the planner still made an incorrect runtime decision. In the Content Summarizer case, the graph appears to support both auditability and actual task completeness. The functional gain from 6/8 to 8/8 suggests that forcing the planner to instantiate assumptions, risks, and policy constraints can improve not just explanation quality but also design coverage.

The intrinsic graph metrics for this project also suggest that iterative graph refinement matters. Across saved graph versions, node recall increases from 0.38 in the first extracted graph to 0.82 by version four, while edge precision rises from 0.55 to 1.00 after the initial iteration and remains there. This progression supports the broader claim of the paper that planning graphs benefit from maintenance and revision over time rather than one-shot extraction alone.

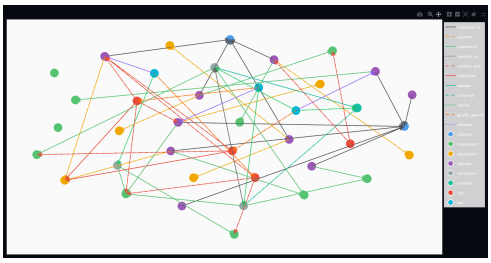

Figure 2: Decision graph for the Content Summarizer under Condition B. The graph contains 36 nodes and 54 edges extracted from two planning exchanges.

5.3 Strengths, weaknesses, and error modes

The strongest result so far is consistency of structural improvement. The graph representation reliably increases explicitness of rationale, alternatives, risks, and assumptions. This is already valuable for software planning because teams often need inspectable reasoning even when the final design is still under debate.

The main weakness is that the benchmark evidence presented here is still narrow. Our strongest reported evidence is a single detailed case study rather than a broad completed benchmark. Another weakness is that Condition B currently depends on prompt-based extraction and a still-evolving feedback mechanism. The system can therefore preserve and expose reasoning better than it can yet guarantee better reasoning from the outset.

6 Conclusion and Future Work

This project explores a practical way to make AI-assisted software planning more persistent, inspectable, and self-critical. By extracting a typed decision graph from planning conversations and documents, then maintaining that graph over time, we make it possible to trace assumptions, record rationale, surface risks, and compute the blast radius of changing decisions. The current Job Copilot case study suggests that graph augmentation substantially improves structural quality and can also modestly improve functional planning quality.

The work nevertheless remains early-stage. Our evaluation suite is still modest, the retrieval-only baseline is not yet implemented, and the iterative feedback loop needs a stronger end-to-end realization. The most promising next steps are to finish the remaining evaluation tasks, add the RAG comparison, strengthen graph re-injection during planning, and expand the prefix-tuning experiments for domain-specific extraction. More broadly, we

believe the project shows that better AI planning support may come not only from larger models, but from better external representations of what the model has already decided, assumed, and risked.

7 Group Member Contributions

The project was divided into several tightly connected workstreams, and each group member took primary ownership over a subset of the system while also contributing to shared design decisions.

Naman Sogani helped lead the overall project framing and the initial formalization of the decision-graph representation. In particular, he contributed to defining the node and edge taxonomy, clarifying which planning concepts should be first-class graph objects, and shaping how the extraction stage should map free-form planning conversations into a typed schema (15% contribution).

Mari Garey worked closely on the schema and extraction side as well, helping refine the representation so that it balanced expressiveness with practical extractability from LLM outputs. Together, this work established the core structure on which the rest of the project depended (15% contribution).

Elena Motonishi and Sreemae Konduru focused primarily on the data and analysis pipeline. Their contributions included identifying useful external sources for software-planning language, collecting and organizing artifacts from GitHub issues, Stack Overflow discussions, and public post-mortem documents, and helping characterize what kinds of signals in those documents correspond to assumptions, risks, decisions, and dependencies (15% contribution each).

Ashvin Sehgal concentrated on the dynamic aspects of the planning graph: maintaining it over time, merging updates, handling invalidation, and supporting blast-radius analysis when upstream assumptions or decisions change. This contribution was central to the project's claim that the graph should function as a living planning artifact rather than a one-time summary (20% contribution).

Omkar Khade contributed to the evaluation framing and broader system integration, including the design of comparison conditions, support for rubric-based analysis, and connection of the different subsystems into a coherent project narrative. He also helped shape the experimental story presented in the case study and supported the interpretation of the results (20% contribution).

Although responsibilities were distributed by subsystem, the project was collaborative throughout. All team members participated in brainstorming the research question, discussing tradeoffs in the graph design, reviewing intermediate outputs, shaping the presentation, and refining the overall argument of the report. The final system and write-up therefore reflect both individual ownership of technical components and repeated group-level iteration on how those components fit together into a single research contribution.

References

Bowen Jin, Chulin Xie, Jiawei Zhang, Kashob Kumar Roy, Yu Zhang, Suhang Wang, Yu Meng, and Jiawei Han. 2024. Graph Chain-of-Thought: Augmenting Large Language Models by Reasoning on Graphs. *arXiv preprint arXiv:2404.07103*.

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. *arXiv preprint arXiv:2503.09516*.

Yuanjie Lyu, Zihan Niu, Zheyong Xie, Chao Zhang, Tong Xu, Yang Wang, and Enhong Chen. 2024. Retrieve-Plan-Generation: An Iterative Planning and Answering Framework for Knowledge-Intensive LLM Generation. In *Proceedings of EMNLP 2024*.

Costas Mavromatis and George Karypis. 2025. GNN-RAG: Graph Neural Retrieval for Efficient Large Language Model Reasoning on Knowledge Graphs. In *Findings of ACL 2025*.

Hui Wei, Zihao Zhang, Shenghua He, Tian Xia, Shijia Pan, and Fei Liu. 2025. PlanGenLLMs: A Modern Survey of LLM Planning Capabilities. In *Proceedings of ACL 2025*.